

LAB1 FPGA1

실험 후 보고서

실험자 : 권진혁

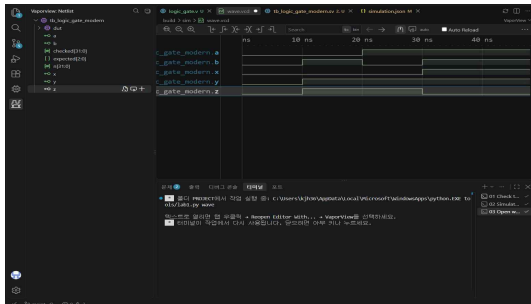
학번 : 2023440007

학과 : 전자전기컴퓨터공학부

실험 일시 : 2026. 09. 14

1. AND, OR, XOR 논리 게이트

· VS Code



· Vivado

Name	Value	0.000 ns	5.000 ns	10.000 ns	15.000 ns	20.000 ns	25.000 ns	30.000 ns	35.000 ns
a	1								
b	1								
x	1								
y	1								
z	0								
> #checked[1]	0000004	0000000	0000001	0000002	0000003				
> #checked[2]	0000004	0000000	0000001	0000002	0000003				
> #expected[2]	5	0		3		6			

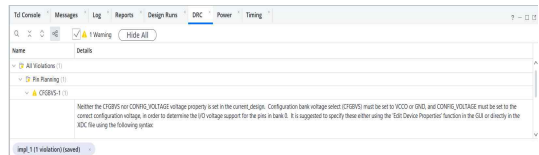
· 결과 비교

VS Code의 Vapor View 파형과 Vivado 시뮬레이션 파형을 비교한 결과, 입력 조합 (a,b) = (00), (01), (10), (11)에 대해 출력 x, y, z의 변화가 모두 동일하게 나타났다. x는 a 신호와 동일하게 동작하였고, y는 OR 게이트의 특성에 따라 둘 중 하나 이상이 1일 때 High가 되었으며, z는 XOR 게이트의 특성에 따라 두 입력이 서로 다를 때만 High가 되었다. Vivado의 expected 값과 checked 값 또한 일치하여 TB가 정상적으로 동작함을 확인하였다. 따라서 VS Code 사전 검증 결과와 Vivado 시뮬레이션 결과는 서로 일치하며 설계가 올바르게 구현되었음을 확인하였다.

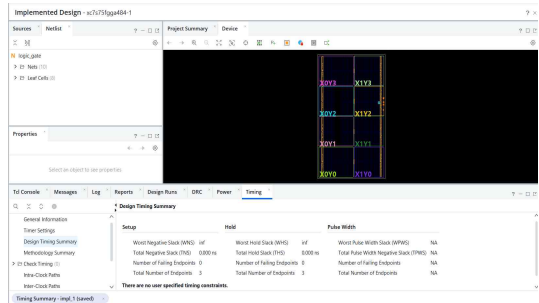
· Hash 코드

Algorithm	Hash	Path
SHA256	AT08C905A4897B5D5ED8222C9E36AC0976380926C5E7AC146971A3898BF01	C:\Users\sumin\lab1_01_logic_...

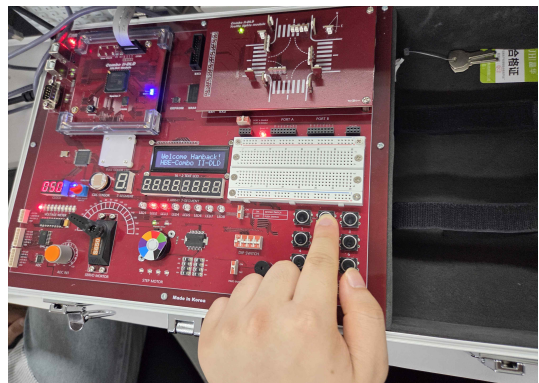
· DRC

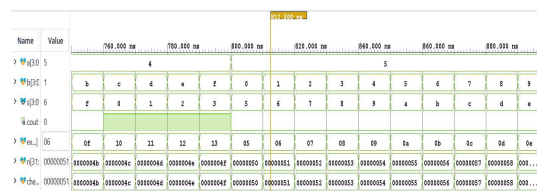


· Timing



· FPGA 보드 사진





· 결과 비교

VS Code의 사전 시뮬레이션 결과와 Vivado 시뮬레이션 결과를 비교한 결과, 모든 입력 조합에서 출력 S와 Cout이 동일하게 나타났다. 출력 S는 항상 A+B의 연산 결과와 일치하였으며, 결과가 4비트 범위를 초과하는 경우 Cout이 정상적으로 발생하였다. 또한 TB의 expected 값과 실제 출력값이 모두 일치하여 설계가 올바르게 구현되었음을 확인하였다. 따라서 VS Code에서 수행한 기능 검증 결과가 Vivado에서도 동일하게 재현되었으며, 4비트 가산기가 정상 동작함을 검증하였다.

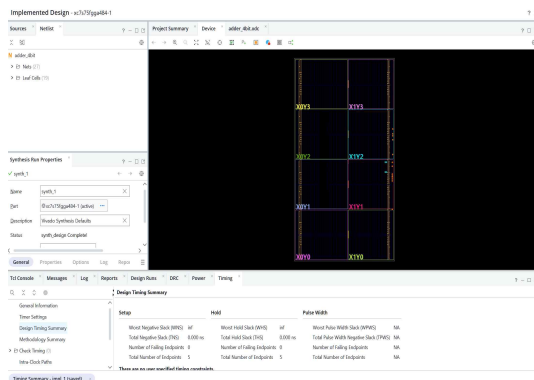
· Hash 코드

Algorithm	Hash	Path
SHA256	4A1C98B94EB38B02B3D676C6A2EA9CA983BDA5C43F72A73A5575E65186385	D:\Engineering\Vivado_Work\La...

· DRC



· Timing



· FPGA 보드 사진



4. 4비트 감산기

· VS Code



· Vivado



· 결과 비교

VS Code의 사전 시뮬레이션 파형과 Vivado 시뮬레이션 파형을 비교한 결과, 모든 입력 조합에서 출력 D와 Borrow 신호가 동일하게 나타났다. 감산 결과 D는 항상 A-B의 계산 결과와 일치하였으며, 피감수보다 감수가 큰 경우 Borrow 신호가 정상적으로 발생하였다. 또한, TB의 expected 값과 실제 출력값이 모두 일치하여 설계가 올바르게 구현되었음을 확인하였다. 따라서 VS

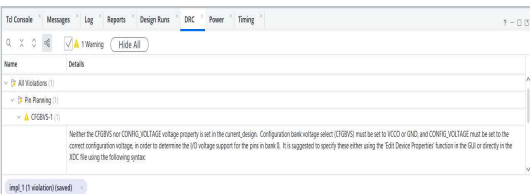
Code에서 검증한 4비트 감산기 동작이 Vivado에서도 동일하게 재현되었으며, 회로가 정상적으로 동작함을 확인하였다.

· Hash 코드

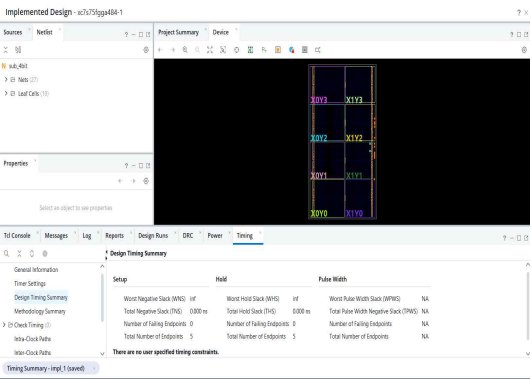
```
PS C:\WINDOWS\system32> Get-FileHash C:\Users\sumin\lab1_04_subtractor\sub_4bit_runs\impl_1\sub_4bit.bit -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	008A51A9798FF14C3E01586632084CF6081BE37C5C0B466FE97271E459FF	C:\Users\sumin\lab1_04_subtra...

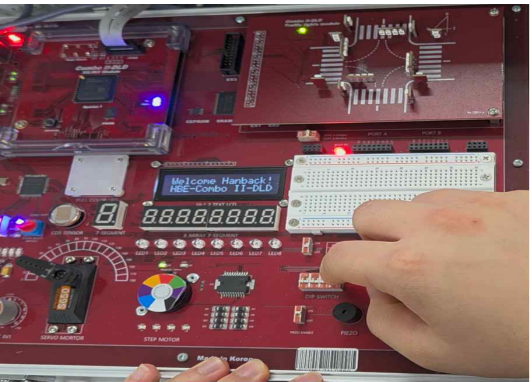
· DRC



· Timing



· FPGA 보드 사진

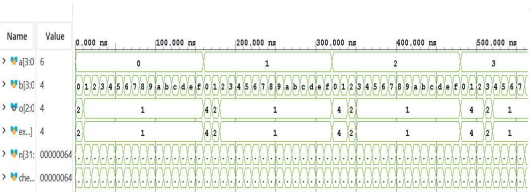


5. 4비트 크기 비교기

· VS Code



· Vivado



· Hash 코드

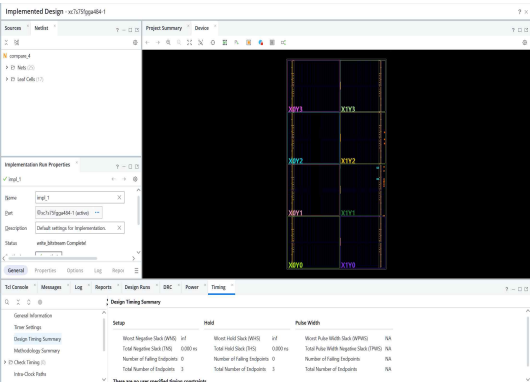
```
PS C:\Users\josun> Get-FileHash D:\Engineering\Vivado_Work\lab1\compare_4\compare_4_runs\impl_1\compare_4.bit -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	06BF8B321D60F1C9FCB2B71BF06E8F87F1D3A6B776886E69A1FE8B47B1CA74A	D:\Engineering\Vivado_Work\La...

· DRC



· Timing



· 결과 비교

VS Code(VaporView)에서 수행한 사전 시뮬레이션 결과와 Vivado 시뮬레이션 결과를 비교한 결과, 4비트 크기 비교기의 출력이 모든 입력 조합에서 동일하게 나타났다. 비교기 회로는 두 개의 4비트 입력 A와 B의 크기를 비교하여 $A > B$, $A = B$, $A < B$ 상태를 출력한다.

본 실험에서 출력 $o[2:0]$ 은 다음과 같이 정의된다.

- 001(1) : $A < B$
- 010(2) : $A = B$
- 100(4) : $A > B$

VS Code 파형에서는 출력값이 1, 2, 4로 변화하는 모습을 확인할 수 있으며, Vivado 파형에서도 동일한 값이 출력되고 있다. 또한 expected와 실제 출력 $o[2:0]$ 이 항상 같게 나타나 TB 검증이 정상적으로 수행되었음을 확인할 수 있다.

· A = 0일 때

Vivado 파형을 보면 B가 0부터 F까지 증가한다. 이는 VS Code 파형에서도 동일하게 나타난다. 처음에는 출력이 2($A = B$)이며, 이후 B가 증가하면 출력이 1($A < B$)로 유지된다.

A	B	예상 결과	출력 o
0	0	$A = B$	2
0	1~F	$A < B$	1

· A = 1일 때

Vivado 파형에서는 출력이 4 - 2 - 1 순서로 변화하며, VS Code에서도 동일한 패턴이 관찰된다.

A	B	예상 결과	출력 o
1	0	$A > B$	4
1	1	$A = B$	2
1	2~F	$A < B$	1

· A = 2일 때

두 시뮬레이션 모두 동일한 결과를 보여준다.

A	B	예상 결과	출력 o
2	0,1	$A > B$	4
2	2	$A = B$	2
2	3~F	$A < B$	1

· Vivado Cursor 위치 분석

Vivado 사진의 현재 값은 다음과 같다.

- A = 6
- B = 4
- o = 4

크기를 비교하면 ($6 > 4$)이므로 ($A > B$) 상태가 되어 출력은 $o = 100_2 = 4$ 가 된다. VS Code 파형에서도 A = 6, B = 4에 해당하는 구간에서 출력이 4로 나타나 동일한 결과를 확인할 수 있다.

· 출력값 해석

비교기 출력의 의미는 다음과 같다.

출력값(o)	의미
1 (001)	$A < B$
2 (010)	$A = B$
4 (100)	$A > B$

Vivado 파형에서도 출력 신호가 반복적으로 4 - 2 - 1 형태를 보이는 이유는 B가 증가하면서 ($A > B$) - ($A = B$) - ($A < B$) 상태로 변하기 때문이다. VS Code 파형에서도 동일하게 출력이 4, 2, 1 형태로 변화하는

것을 확인할 수 있다.

· TB 검증 결과

Vivado 파형에서

- o[2:0] = 4
- expected = 4
- checked = 00000064
- n = 00000064

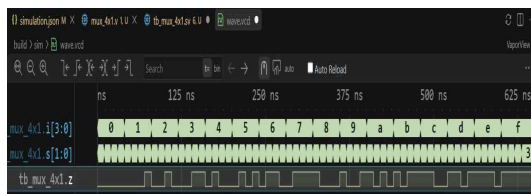
로 표시된다. 실제 출력과 기대값이 일치하므로 TB가 정상적으로 PASS하였음을 확인할 수 있다. 이러한 결과는 모든 입력 조합에 대해 동일하게 나타난다.

· FPGA 보드 사진

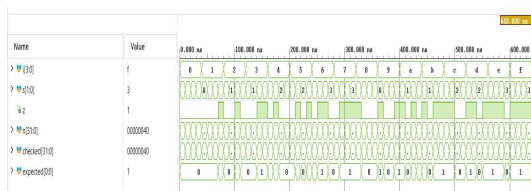


6. 4:1 MUX

· VS Code



· Vivado

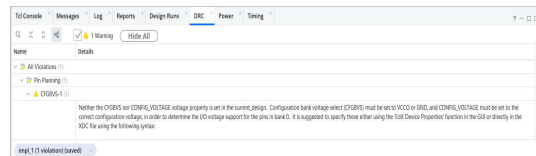


· Hash 코드

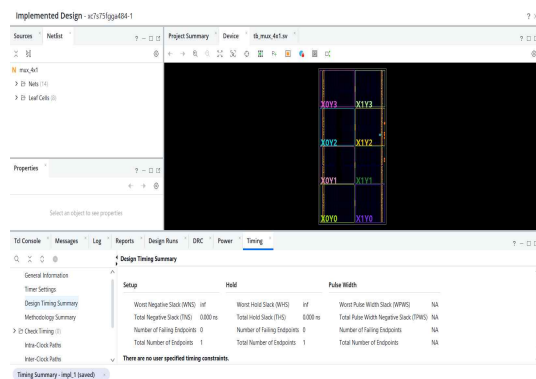
```
PS C:\WINDOWS\system32> Get-FileHash C:\Users\sumin\lab1_06_mux4to1_mux_4x1_run\impl_1\mux_4x1.bit -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	58D8D034436E565C6E4DABE7D17F8E178FF8E8206586A98D02B8FC98ACEF8	C:\Users\sumin\lab1_06_mux4to1_mux_4x1_run\impl_1\mux_4x1.bit

· DRC



· Timing



· 결과 비교

VS Code 사전 시뮬레이션과 Vivado 시뮬레이션을 비교한 결과, 모든 입력 조합에서 출력 z가 동일하게 동작하였다. 선택 신호 s[1:0]는 항상 지정된 입력 비트(i0~i3) 중 하나를 정확하게 선택하였으며, 실제 출력값은 TB의 expected 값과 모두 일치하였다. 따라서 4:1 Multiplexer 회로가 정상적으로 구현되었으며, VS Code에서 검증한 동작이 Vivado에서도 동일하게 재현됨을 확인하였다.

· 파형 비교 분석

VS Code(VaporView)에서 수행한 사전 시뮬레이션 결과와 Vivado 시뮬레이션 결과를 비교한 결과, 4:1 Multiplexer의 출력 z가 선택 신호 s[1:0]에 따라 동일하게 동작함을

확인하였다.

4:1 MUX의 동작은 다음과 같다.

선택신호 s[1:0]	출력 z
00	i[0]
01	i[1]
10	i[2]
11	i[3]

즉, 선택신호의 값에 따라 입력 i[3:0] 중 하나가 출력 z로 전달된다. VS Code 파형에서는 입력 i가 0부터 F까지 증가하고 선택 신호 s가 00→01→10→11로 반복적으로 변화하며, 출력 z가 해당 비트 값을 선택하여 출력하는 모습을 볼 수 있다. Vivado 파형에서도 동일한 입력 조합에 대해 같은 출력값이 나타나 두 시뮬레이션 결과가 일치함을 확인하였다.

· 입력 i = 0 (0000)

s	선택 입력	출력 z
00	i0=0	0
01	i1=0	0
10	i2=0	0
11	i3=0	0

모든 비트가 0이므로 출력은 항상 0이다.

· 입력 i = 1 (0001)

s	선택 입력	출력 z
00	i0=1	1
01	i1=0	0
10	i2=0	0
11	i3=0	0

선택 신호가 00일 때만 출력이 1이 된다.

· 입력 i = 2 (0010)

s	선택 입력	출력 z
00	i0=0	0
01	i1=1	1
10	i2=0	0
11	i3=0	0

선택 신호가 01일 때만 출력이 1이 된다.

· 입력 i = 4 (0100)

s	선택 입력	출력 z
00	i0=0	0
01	i1=0	0
10	i2=1	1
11	i3=0	0

선택신호가 10일 때만 출력이 1이 된다.

· 입력 i = 8 (1000)

s	선택 입력	출력 z
00	i0=0	0
01	i1=0	0
10	i2=0	0
11	i3=1	1

선택 신호가 11일 때만 출력이 1이 된다.

· Vivado Cursor 위치 분석

Vivado 파형의 현재 커서(640ns)에서는

- i [3:0] = F(1111)

- s [1:0] = 3(11)

- z = 1이다.

MUX 동작에 따라 s = 11이면 i3 선택 - i3 = 1 - z = 1이 되어 예상 결과와 정확히 일치한다. 또한,

- expected = 1

- z = 1

- checked = 00000040

- n = 00000040으로 나타나 실제 출력과 기댓값이 일치함을 확인할 수 있다.

· FPGA 보드 사진

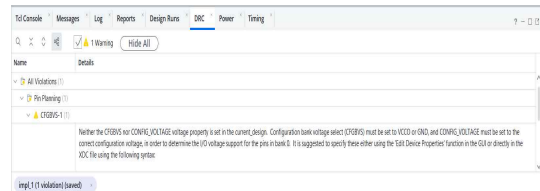


expected 값과 실제 출력값이 모두 일치하였다. 따라서 설계한 1×8 DE-MUX회로가 정상적으로 구현되었으며, VS Code에서 검증한 결과가 Vivado에서도 동일하게 재현됨을 확인하였다.

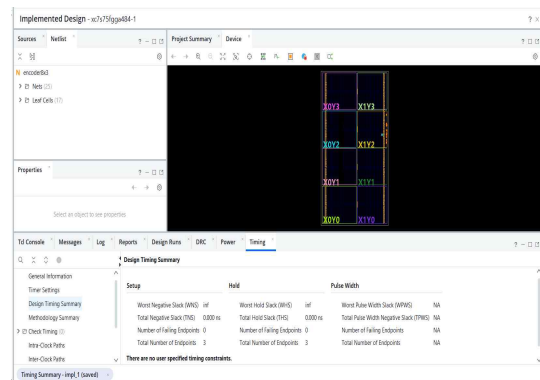
· Hash 코드

```
PS C:\WINDOWS\system32> Get-FileHash C:\Users\sumin\lab1_08_encoder\to3\encoder0x3_runs\inp_1_encoder0x3.bit -Algorithm SHA256
Algorithm Hash Path
-----
SHA256 4863680b2368ff8c38aef44d89c1fb631b82cc3e38147876d124e85598a36d09 C:\Users\sumin\lab1_08_encode...
```

· DRC

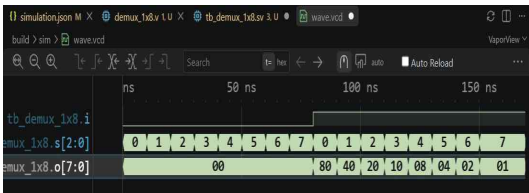


· Timing



7. 1:8 DE-MUX

· VS Code



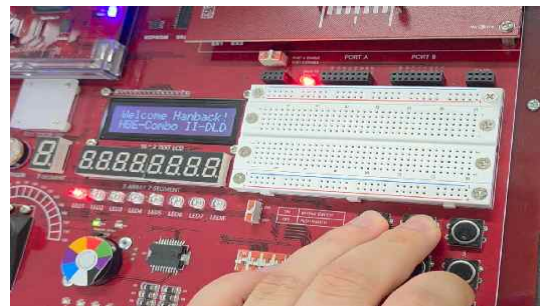
· Vivado



· 결과 비교

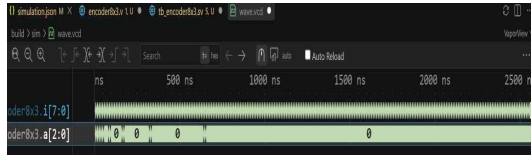
VS Code 사전 시뮬레이션 결과와 Vivado 시뮬레이션 결과를 비교한 결과, 선택 신호의 변화에 따라 입력 신호가 정확히 하나의 출력으로만 전달되는 동일한 동작을 확인하였다. 입력이 0일 때는 모든 출력이 0으로 유지되었으며, 입력이 1일 때는 선택 신호에 의해 지정된 출력만 1이 되고 나머지 출력은 0이 되었다. 또한 One-Hot 출력 패턴이 두 시뮬레이션에서 동일하게 나타났으며,

· FPGA 보드 사진

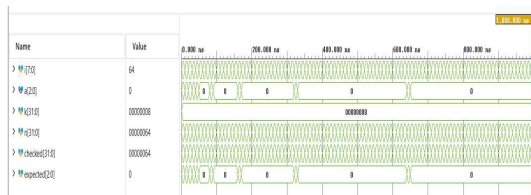


8. 8:3 인코더

· VS Code



· Vivado



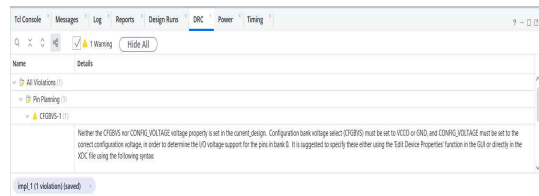
· 결과 비교

VS Code 사전 시뮬레이션 결과와 Vivado 시뮬레이션 결과를 비교한 결과, 모든 입력 패턴에 대해 동일한 출력이 나타났다. 활성화된 입력선의 위치가 정확하게 3비트 이진수로 인코딩되었으며, 출력값은 인코더의 진리표와 완전히 일치하였다. 또한, Vivado의 expected 값과 실제 출력값이 모두 동일하여 TB 검증이 정상적으로 수행됨을 확인하였다. 따라서 설계한 8×3 Encoder 회로는 VS Code와 Vivado 환경 모두에서 정상적으로 동작하며, 기능이 올바르게 구현되었음을 검증하였다.

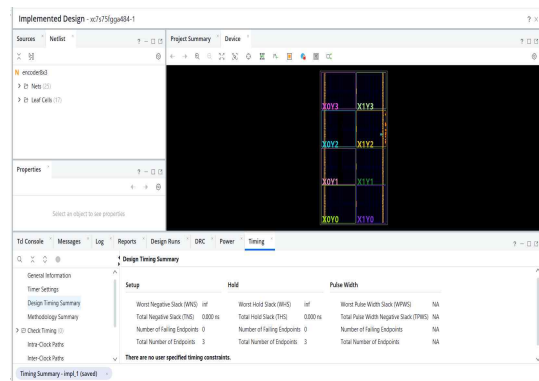
· Hash 코드

```
PS C:\WINDOWS\System32> Get-FileHash C:\Users\sumin\lab1_08_encoder8to3\encoder8x3.runs\impl_1\encoder8x3.bit -Algorithm SHA256
Algorithm Hash Path
-----
SHA256 48b63680d2368ff8c3baef44d98c1f8631882cc3e381478760124e85598a360d9 C:\Users\sumin\lab1_08_encode...
```

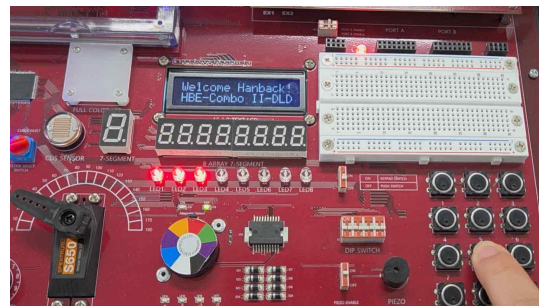
· DRC



· Timing



· FPGA 보드 사진



9. 3:8 인코더

· VS Code



· Vivado

Name	Value	0.000 ns	10.000 ns	20.000 ns	30.000 ns	40.000 ns	50.000 ns	60.000 ns	70.000 ns
b	1								
b	1								
c	1								
W[0]	00	01	02	04	08	10	20	40	80
W[0]	00000000	00000000	00000002	00000004	00000008	00000010	00000020	00000040	00000080
W[checked]	00000000	00000000	00000002	00000004	00000008	00000010	00000020	00000040	00000080
W[expected]	00	01	02	04	08	10	20	40	80

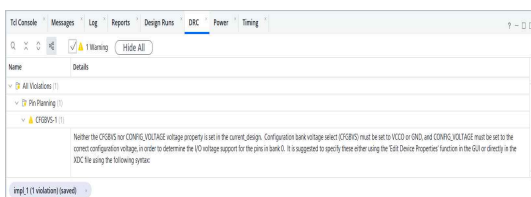
· 결과 비교

VS Code 사전 시뮬레이션 결과와 Vivado 시뮬레이션 결과를 비교한 결과, 모든 입력 조합에 대해 동일한 출력이 생성되었다. 입력값 000~111에 따라 출력은 01→02→04→08→10→20→40→80 순으로 변화하였으며, 항상 하나의 출력선만 활성화되는 One-Hot 특성이 유지되었다. 또한 실제 출력값과 expected 값이 모두 일치하여 TB 검증이 성공적으로 수행되었음을 확인하였다. 따라서 설계한 3×8 Decoder 회로가 정상적으로 구현되었으며, VS Code와 Vivado 환경에서 동일한 동작을 수행함을 검증하였다.

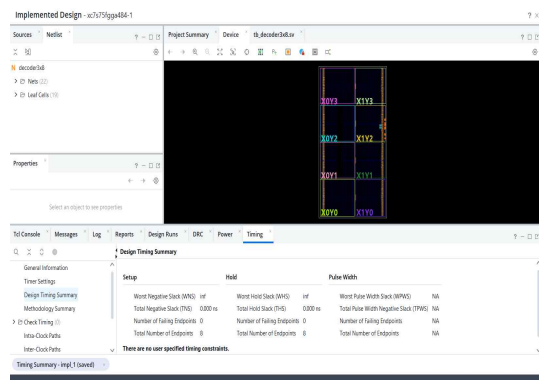
· Hash 코드

PS C:\WINDOWS\System32> Get-FileHash C:\Users\sumin\lab1_09_decoder\ta0\decoder3x8_run\lab1_1\decoder3x8.bit -Algorithm SHA256
Algorithm Hash Path
SHA256 F69C94C968173D1C43C1B798DA9E88BF4F016A53889C0C92AAEAF82 C:\Users\sumin\lab1_09_decode...

· DRC



· Timing



· FPGA 보드 사진

10. 7-세그먼트 디코더

· VS Code

ns	25 ns	50 ns	75 ns	100 ns	125 ns	150 ns
seg_data[3:0]	0	1	2	3	4	5
seg_data[7:0]	fc	60	da	f2	66	b6

· Vivado

Name	Value	0.000 ns	10.000 ns	20.000 ns	30.000 ns	40.000 ns	50.000 ns	60.000 ns	70.000 ns
b	1	0	1	2	3	4	5	6	7
b	1	0	1	2	3	4	5	6	7
c	1	0	1	2	3	4	5	6	7
W[0]	00	01	02	04	08	10	20	40	80
W[0]	00000000	00000000	00000002	00000004	00000008	00000010	00000020	00000040	00000080
W[checked]	00000000	00000000	00000002	00000004	00000008	00000010	00000020	00000040	00000080
W[expected]	00	01	02	04	08	10	20	40	80

· 결과 비교

VS Code 사전 시뮬레이션 결과와 Vivado 시뮬레이션 결과를 비교한 결과, 입력값 0~F의 모든 경우에서 동일한 Segment 출력 패턴이 생성되었다. 입력값이 변경될 때마다 출력 seg_data가 예상한 7-Segment 코드와 정확히 일치하였으며, Vivado의 expected 값과 실제 출력값 또한 모두 동일하였다. 따라서 설계한 Seven Segment Decoder가 정상적으로 동작하며, VS Code

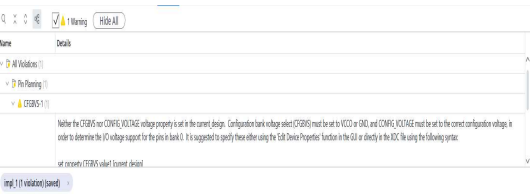
에서 확인한 결과가 Vivado 환경에서도 동일하게 재현됨을 검증하였다. 또한 모든 테스트 케이스가 PASS하여 회로가 올바르게 구현되었음을 확인하였다.

· Hash

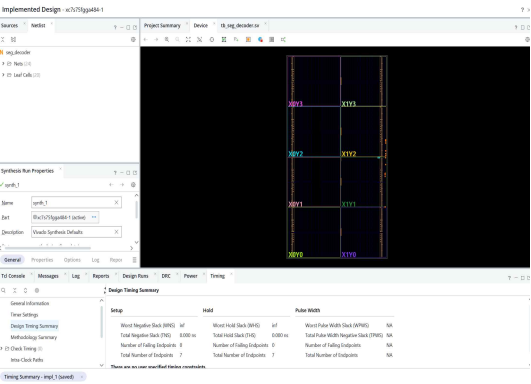
```
PS D:\Desktop> Get-FileHash "D:\Engineering\Vivado_Work\Lab1\seq_decoder\seq_decoder_run\seq_decoder.t11" -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	7C93262E38AF325HELCC89C840305E1845CC47DA1BC7F08975C972999389945B	D:\Engineering\Vivado_Work\La...

· DRC



· Timing



11. 참고 문헌

· 전자전기컴퓨터설계실험 II 교안

12. Git-Hub 주소

<https://github.com/UnnamedDays/ECE-II-Project>